CS-117: Introduction to ICT

Lab 11 - Git & GitHub: Version Control and Collaborative Development

3-hour practical lab | Git CLI + GitHub | Reuse your Lab 10 Python program

STUDENT COPY

1. What You Will Learn

In Lab 10 you created and debugged a Python program. In this lab you will learn how developers keep a trustworthy history of that program, work on changes safely, share them through GitHub, review each other's work, and resolve a simple conflict.

Learning Flow
Initialize -> Track -> Commit -> Inspect -> Share -> Branch -> Review -> Merge -> Resolve -> Explain

2. Git and GitHub Are Not the Same

Git	GitHub
Version-control software that runs on your computer.	Online platform that hosts Git repositories and adds collaboration features.
Tracks versions, branches and commits.	Provides remote hosting, Issues, Pull Requests, reviews and web history.

3. Before You Start

- Sign in to the GitHub environment specified by your instructor.
- Confirm Git is installed by running: git --version
- Have a copy of your Lab 10 Python file ready, preferably storage_estimator.py.
- Never place passwords, tokens, private keys, or confidential information in the repository.

4. Essential Vocabulary

Term	Meaning
Repository	A project and its Git change history.
Working tree	Files you are currently editing.
Staging area	Changes selected for the next commit.
Commit	A recorded project snapshot with a message.
Branch	A separate line of development.
Remote	A connection to another Git repository, commonly on GitHub.
Push	Send local commits to the remote.
Pull	Bring remote changes into the current local branch.
Pull Request	Proposal to review and merge one branch into another.

5. Session Plan

Time	Activity
0:00-0:15	Git/GitHub concepts
0:15-0:40	Create repository and first commits

Time	Activity
0:40-1:00	Status, diff, history, README and .gitignore
1:00-1:20	Push to GitHub and synchronize
1:20-1:30	Break
1:30-1:55	Issue and feature branch
1:55-2:20	Pull Request, review and merge
2:20-2:40	Merge conflict
2:40-2:50	Final synchronization and history
2:50-3:00	Exit task / viva

Task 1 - Create Your Local Repository

```
mkdir Lab11_Git
cd Lab11_Git
# Copy storage_estimator.py into this folder
git init
git status
```

If your instructor asks you to configure your Git identity, use your real academic name and email:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.edu"
```

Stage your Python file and inspect the status before committing:

```
git add storage_estimator.py
git status
```

Create the first commit:

```
git commit -m "Add initial storage estimator"
git log --oneline
```

Think Before Moving On

What is the difference between adding a file to the staging area and committing it? Record your explanation on the evidence sheet.

Task 2 - Change, Compare and Commit

Improve the Python file in one small meaningful way. Save it, then inspect the change before staging:

```
git status
git diff
```

Now stage and commit the improvement:

```
git add storage_estimator.py
git commit -m "Improve storage estimator output"
git log --oneline
```

Task 3 - Add README and .gitignore

Create README.md containing: project title, purpose, how to run the program, example input/output, and author/group information as instructed.

Create .gitignore with common Python-generated files, for example:

```
__pycache__/
```

```
*.pyc
.venv/
venv/
git add README.md .gitignore
git commit -m "Add project documentation and ignore rules"
```

Commit Messages
Use messages that describe the change. Avoid vague messages such as "update", "final", or "stuff".

Task 4 - Connect to GitHub

Create an empty GitHub repository using the naming convention provided by your instructor. Copy its repository URL.

```
git branch -M main
git remote add origin <repository-url>
git remote -v
git push -u origin main
```

Open the repository in your browser. Confirm your files and commit history are visible.

Remember
Commit records a version locally. Push sends local commits to the remote repository. They are different actions.

Task 5 - Issue -> Feature Branch -> Change

Create or use the Issue assigned by your instructor. Example: "Add a GB conversion line to the storage estimator."

```
git switch -c feature/gb-output
git branch
```

Modify the Python program only for the assigned feature. Test it before committing.

```
python storage_estimator.py
git diff
git add storage_estimator.py
git commit -m "Add GB conversion output"
git push -u origin feature/gb-output
```

Task 6 - Pull Request and Review

1. Open a Pull Request from your feature branch to main.
2. Write a short description explaining what changed and how you tested it.
3. A partner/group member reviews the PR and leaves at least one specific comment.
4. Respond to the review. Make an additional commit if a valid improvement is needed.
5. Merge the Pull Request using the method specified by your instructor.

Then synchronize locally:

```
git switch main
git pull
git log --oneline --graph --decorate --all
```

Task 7 - Resolve a Controlled Merge Conflict

Your instructor will guide you in creating a simple conflict in README.md. Git may insert markers such as:

```
<<<<<<< HEAD
Storage Estimator - Main Version
=======
Storage Estimator - Feature Version
>>>>>>> feature/title-change
```

Read both sides. Decide what the final text should be. Remove the markers and unwanted text, then complete the resolution:

```
git status
git add README.md
git commit -m "Resolve README title conflict"
git status
```

> **Important**
> A merge conflict is Git asking you to make a decision. Do not delete conflict markers blindly without understanding the competing changes.

Task 8 - Final Repository Check

```
git status
git branch
git remote -v
git log --oneline --graph --decorate --all
```

Your working tree should be clean. You should be able to explain your main branch, feature work, remote, meaningful commits, Pull Request, and conflict-resolution commit.

Troubleshooting

Problem	What to Check
Not a git repository	Are you in the correct folder?
Nothing to commit	Was the file saved? Is it already committed or ignored?
Push rejected	Remote may have newer work. Inspect and pull safely; do not force-push unless instructor explicitly directs it.
Authentication error	Use the supported sign-in method. Never paste credentials into evidence.
Wrong branch	Run git branch and git status before changing more files.
Conflict	Read both versions, create the correct final file, stage it, and commit the resolution.

Individual Evidence

- Repository name and remote.
- First commit short hash and message.
- One git status interpretation.
- One git diff you inspected.
- Feature branch name.
- Issue/PR evidence.
- Specific review comment.

- Conflict diagnosis and resolution.
- Randomized live exit task with instructor initials.

Required Repository State / Submission

- Submit the GitHub repository URL through the LMS as instructed.
- Repository contains storage_estimator.py (or approved Lab 10 program), README.md and .gitignore.
- At least 4 meaningful commits.
- At least one feature branch was used.
- At least one Issue and one Pull Request with review evidence.
- Controlled conflict resolved.
- Individual evidence sheet completed.

Final Concept

What You Should Remember

Git is not just a place to store files. It records meaningful changes, supports parallel work, and helps teams review and integrate improvements safely.

Assessment - 10 Marks

Component	Marks
Git fundamentals and local history	1.5
GitHub remote workflow	1.5
README, .gitignore and professional repository practice	1.0
Branching and feature development	1.5
Issue, Pull Request and review	1.5
Conflict diagnosis and resolution	1.0
Individual evidence, exit task and viva	2.0
Total	10.0

Analytic Evaluation Rubric

Criterion	Wt.	Excellent	Good	Satisfactory	Needs Improvement
Git fundamentals & local history	1.5	Correctly uses status, add, commit, log and diff; explains working tree, staging and commit history.	Mostly correct with minor prompting or small history issues.	Completes core commands with guidance; explanations basic.	Cannot reliably stage/commit or explain repository state.
GitHub remote workflow	1.5	Connects remote, pushes/pulls safely, and clearly distinguishes local vs remote.	Workflow succeeds with minor assistance.	Completes basic push/pull with substantial guidance.	Cannot connect/synchronize repository or confuses local and remote.
README, .gitignore & professional practice	1.0	Clear README, appropriate .gitignore and meaningful commit messages/files.	Minor omissions or inconsistencies.	Minimum documentation present with guidance.	Repository poorly documented or includes avoidable files.
Branching & feature development	1.5	Creates/switches branch, makes scoped change, verifies and merges correctly.	Mostly correct with minor assistance.	Feature workflow completed with significant prompting.	Cannot manage branch workflow reliably.
Issue, Pull Request & review	1.5	Uses issue-to-branch-to-PR workflow; review comment is specific and merge is justified.	Workflow complete with minor gaps.	Basic PR/review completed with guidance.	Little evidence of collaborative workflow.
Conflict diagnosis & resolution	1.0	Explains conflict, resolves intentionally, tests result, stages and commits resolution.	Correct resolution with minor prompting.	Resolves with substantial instructor guidance.	Deletes/guesses without understanding or leaves repository conflicted.
Individual competence & viva	2.0	Performs live Git task independently and accurately explains repository state and commands.	Minor prompting; explanation mostly accurate.	Basic competency with significant prompting.	Cannot independently demonstrate or explain core workflow.

Individual Competence Rule

A polished group repository cannot compensate for weak individual understanding. Full individual marks require the student to demonstrate and explain core Git operations live.